

# Memoria — Prototipo nativo en Ionic

## Red de Donación de Alimentos · Grupo GCS-08

Asignatura: Gestión de Calidad de Software (34041) · Repositorio: <https://github.com/eap59-ua/food-donation-backend> (rama `feature/frontend-Ionic`)

---

### 1. Qué hemos hecho

A partir de las conclusiones del test de usuarios sobre los dos prototipos tempranos (FoodDonationA y FoodDonationB), hemos implementado el **prototipo nativo en Ionic** tomando como base el **Prototipo A**, por ser el que mejor rendía en las tareas críticas (aceptar solicitudes T2 y compartir donaciones T5), con mayor compleción global y una experiencia percibida (EE) significativamente superior. Del Prototipo B incorporamos sus aciertos estéticos (paleta limpia, más espacios en blanco) para mejorar la percepción de marca (donde B ganaba en NPS) y corregimos sus problemas críticos de navegación y feedback.

El entregable se compone de: - **Backend** — API REST en FastAPI ya existente, reutilizada como núcleo lógico. - **Frontend nativo Ionic** — RedDonacion-Ionic, reconstruido desde cero siguiendo la estructura de los PDF de prácticas (módulos Angular con *lazy loading*). - En el repositorio queda también `frontend-Ionic` (primera versión generada, con problemas de enrutado de botones) como histórico; **el prototipo válido es RedDonacion-Ionic**.

### 2. Funcionalidades del prototipo

Flujo completo de la aplicación, todas las pantallas conectadas contra la API: - **Landing** pública y **registro / inicio de sesión** (roles DONANTE, RECEPTOR, ONG, ADMIN). - **Dashboard** con feed de donaciones disponibles, histórico de aportaciones y solicitudes pendientes. - **Crear / editar donación** (`create-donation`). - **Listado y detalle de donaciones** (`donations-list`, `donation-detail`). - **Gestión de solicitudes**: crear, ver, **aceptar** y **rechazar** (`my-requests`). - Acceso protegido por **guards** (`AuthGuard / GuestGuard`) e **interceptor JWT** que adjunta el token a cada petición; token persistido con `@ionic/storage`.

### 3. Backend (cómo está hecho)

- **FastAPI (Python 3.11)** con **arquitectura hexagonal**: `domain` (entidades y enums), `application` (servicios y DTOs), `infrastructure` (modelos SQLAlchemy y BD) y `presentation` (rutas API v1).
- **PostgreSQL asíncrono** vía SQLAlchemy + `asyncpg`.
- **Autenticación JWT** (HS256) con hashing **bcrypt** y control de acceso por rol.
- Endpoints: `/api/v1/auth` (`register`, `login`, `me`), `/api/v1/donations`, `/api/v1/requests`.
- Despliegue con **Docker Compose** (servicios `api`, `db` `Postgres 16`, `frontend`).

**Estado verificado**: el backend arranca y funciona correctamente (`health`, `registro`, `login` con emisión de JWT y creación de tablas en el arranque). El error de arranque reportado durante el desarrollo era de entorno Docker (volumen `pgdata` previo o puerto ocupado), no del código. Solución: `docker compose down -v && docker compose build --no-cache && docker compose up`.

### 4. Frontend nativo (cómo está hecho)

- **Ionic 8 + Angular 20**, módulos con **lazy loading** por página y enrutado con `PreloadAllModules`.
- **Tema** portado del Prototipo A (React/Vite): paleta terracota (`#b8602a`) + verde oliva (`#7aab46`) sobre fondo blanco cálido, tipografía **Inter**; variables `--rd-*` en `src/theme/variables.scss`.
- **Capacitor** configurado para empaquetado nativo (móvil).

- Configuración de API en `src/environments`: desarrollo → `http://localhost:8005/api/v1`; producción → `/api/v1`.

## 5. Enfoque de desarrollo (metodología)

Hemos usado un **enfoque mixto**: - **Backend + especificación**: desarrollado con **SDD** (Spec-Driven Development), apoyándonos en los artefactos de `specs/` y en la evaluación heurística documentada (`GCS08_SDD_Heurísticas.pdf`). - **Frontend Ionic**: combinación de **vibe coding** asistido por IA (generación rápida de pantallas y servicios) con **refactor manual** para ajustarlo a la estructura por módulos exigida en las prácticas. La primera generación automática (`frontend-Ionic`) falló en el enrutado de botones, por lo que se reconstruyó (`RedDonacion-Ionic`) con control más explícito de la arquitectura. - **Análisis de usabilidad**: dirigido por datos, con el **script oficial de Python** sobre los resultados de `Loop11`.

## 6. Temporalización

| Fase                                   | Periodo                 | Hito                                                   |
|----------------------------------------|-------------------------|--------------------------------------------------------|
| Backend API (FastAPI hexagonal)        | Feb-Mar 2026            | API REST + Docker + tests                              |
| Prototipos tempranos A y B (Figma/web) | Mar 2026                | Dos variantes de frontend                              |
| Test de usuarios (Loop11) y análisis   | Semana Santa (abr 2026) | G8-FoodDonation-DatosProcesados.xlsx + análisis Python |
| Evaluación heurística                  | mar-abr 2026            | GCS08_SDD_Heurísticas.pdf / .pptx                      |
| Prototipo nativo Ionic                 | jun 2026                | RedDonacion-Ionic (base Prototipo A)                   |
| Memoria + vídeo de defensa             | 14-16 jun 2026          | Esta entrega                                           |

## 7. Problemas y soluciones

| Problema                                                                                           | Causa                                                                                 | Solución                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Botones sin respuesta en el primer frontend Ionic                                                  | Enrutado/ <code>RouteReuseStrategy</code> mal configurado en la generación automática | Reconstrucción desde cero con módulos + lazy loading ( <code>RedDonacion-Ionic</code> )                                                                   |
| Backend “no arrancaba” con Docker                                                                  | Entorno: volumen <code>pgdata</code> previo / puerto ocupado                          | <code>docker compose down -v</code> + <code>rebuild</code> ; código verificado correcto                                                                   |
| Fecha de caducidad de donación con zona horaria                                                    | <code>datetime aware</code> vs columna <i>naive</i>                                   | Normalización<br><code>expiration_date.replace(tzinfo=None)</code> en el servicio                                                                         |
| CORS en desarrollo con puertos cambiantes de Ionic<br>Incompatibilidad <code>bcrypt/passlib</code> | Origen variable de <code>ionic serve</code><br>Versiones                              | <code>allow_origin_regex</code> para cualquier <code>localhost:&lt;puerto&gt;</code><br>Fijado <code>bcrypt==4.0.1</code> ;<br>login/registro verificados |

## 8. Cómo ejecutar

# Backend + BD

`docker compose up --build`

# API en `http://localhost:8005`, docs en `/docs`

```
# Frontend nativo
cd RedDonacion-Ionic
npm install
npm start                # ionic serve
```

## 9. Autoría

Grupo **GCS-08**. Integrantes en autores.txt.

Nota de entrega: para reducir espacio se han eliminado `node_modules`, `.angular` y demás paquetes de terceros antes de comprimir, según las instrucciones.